

1 **Feasibility of Field-Derived High-Resolution Event Replay for Traffic-**
2 **Signal Controller Testing**

3 **Shawn Strasser, P.E.**
4 Traffic Signal Operations Engineer
5 Oregon Department of Transportation
6 shawn.strasser12@gmail.com

7 Total manuscript pages: 13

Structured Abstract

Objectives. Traffic-signal controller changes are difficult to test against the breadth and timing of inputs that occur in service. This study asks whether high-resolution events recorded at live field controllers can be converted to controller calls, replayed at test controllers, and used to automatically identify operational differences in the resulting event sequences.

Methods. Approximately 23 hours of detector, pedestrian, and preemption events were collected for each of 25 production signal configurations. The events were converted to National Transportation Communications for Intelligent Transportation Systems Protocol calls and replayed at their recorded time of day to controller emulators. Output event traces were grouped by timestamp and aligned with a modified dynamic time warping procedure using Jaccard distance between discrete event sets. Feasibility was evaluated through a software-release comparison and a same-software timing-parameter intervention.

Findings. In 11 configurations unchanged between two same-software replay campaigns, mean sequence match was 99.6 percent and mean timing match was 97.7 percent, demonstrating that the field-derived replay produced highly repeatable controller outputs. Deliberate trailing-overlap changes were automatically flagged in 11 of 14 affected configurations, with no flags among the 11 unchanged configurations. Comparing software versions 2.15.1 and 2.18.1 produced 19 passes and six review candidates; manual review identified an expected preemption bug fix and transition-logic changes among the divergences.

Novelty. The study combines long-duration field-derived input replay with automated alignment of complete high-resolution output traces across numerous production configurations, extending prior scripted controller testing and controller-in-the-loop research.

Practical Applications. Agencies can use recorded operation as a reusable regression suite for software releases, timing changes, and targeted bug replication before field deployment.

1 INTRODUCTION

2 Traffic-signal controller software is safety-critical operational software, yet the most difficult behaviors to test are
3 often produced by ordinary field complexity: overlapping detector calls, pedestrian service, coordination transitions,
4 and occasional preemption. A bench checklist can verify an isolated function, and microscopic simulation can generate
5 traffic demand, but neither necessarily recreates the exact sequence and timing of controller inputs that previously
6 occurred in service. Manual testing is also poorly suited to reproducing 23 hours of tenth-second events at dozens of
7 configurations.

8 High-resolution controller logs provide another possibility. These logs record detector changes and controller states
9 with subsecond timestamps and have primarily been used to calculate performance measures or diagnose past operation
10 (Sturdevant et al. 2012). The same records can instead be treated as executable test inputs. Detector, pedestrian, and
11 preemption events can be converted into controller calls, delivered to a test controller at their original times, and reused
12 whenever software or timing changes. The output is another high-resolution event trace rather than a small set of
13 manually specified assertions.

14 This paper therefore asks a direct feasibility question:

15 Can high-resolution event logs collected from live field controllers be replayed to test controllers, and can the
16 resulting output sequences be aligned and compared automatically to identify operational differences?

17 The feasibility condition has two parts. First, repeated presentation of the same recorded input must produce output
18 that is deterministic enough to align despite network, logging, and scheduling jitter. Second, the comparison must
19 preserve meaningful divergences rather than warping them away. A method that meets only the first condition would
20 report similarity but miss change; one that meets only the second would overwhelm users with timing noise.

21 The study evaluates both conditions across 25 production signal configurations. About 23 hours of live inputs per
22 configuration were replayed to create a software-version 2.15.1 baseline. The same field inputs were then replayed
23 under version 2.18.1 and compared with the baseline. A second campaign held version 2.18.1 constant while trailing-
24 overlap parameters were modified, providing a known operational intervention. Dynamic time warping (DTW) aligned
25 groups of simultaneous discrete events, and separate sequence and timing measures supported automated screening.

26 The contribution is the evaluated workflow, not any one component. Automated controller testing, National
27 Transportation Communications for Intelligent Transportation Systems Protocol (NTCIP) communication, controller
28 emulation, high-resolution event logging, and DTW all predate this work. The narrower contribution is showing that
29 long-duration, field-derived input histories can function as reusable regression tests and that complete controller-output
30 traces are sufficiently repeatable for automated alignment across many production configurations.

31 PRIOR WORK AND RESEARCH GAP

32 Scripted controller testing

33 The closest prior work is the traffic-controller automated testing system developed by Li et al. and subsequently
34 documented for the Idaho Transportation Department (Li et al. 2008; Ahmed et al. 2010). That system replaced manual
35 suitcase testing with controlled input activation and response verification. XML scripts specified which inputs to
36 activate, when to activate them, and which responses to expect; variants supported controller interface devices, NEMA
37 controller types, and NTCIP communication. The work established that automation improves repeatability and permits
38 stored test definitions and results.

39 Tung’s Florida Department of Transportation work further developed NTCIP-oriented automated testing across traffic-
40 control devices (Tung 2012; 2015). These projects are important precedents: the present study does not claim to invent
41 automated controller tests or NTCIP-based activation. The distinction is the source and scale of the test. Prior systems
42 primarily executed authored functional scripts with predefined expected responses. Here, the input script is derived
43 from an entire day of actual field operation, and the expected response is a previously collected, complete output
44 trace. This changes the testing task from checking selected assertions to comparing emergent behavior under a broad
45 operational sequence.

46 Controller-in-the-loop and high-resolution event research

47 Controller-in-the-loop studies demonstrate that physical controllers, software controllers, and emulators can be
48 connected to traffic simulations. Wang, Tian, and Yang used a virtual controller interface device and NTCIP to connect

1 controllers to simulation, and compared hardware-, software-, and emulator-in-the-loop environments (Wang et al.
2 2021). Their outcome measures were traffic-simulation measures such as queues, delay, and trajectories. The present
3 study does not simulate vehicles or estimate network performance; it supplies controller inputs directly and examines
4 controller event outputs as a software-regression trace.

5 Stevanovic, Klanac, and Radivojevic evaluated high-resolution logging and performance-measure consistency for six
6 controller vendors in a hardware-in-the-loop laboratory (Stevanovic et al. 2017). Their findings reinforce two cautions
7 relevant here: event definitions and logging behavior require normalization, and results from one platform do not
8 automatically transfer to another. Their objective was logger and performance-measure evaluation, not replay of
9 recorded field inputs for release regression.

10 Sequence alignment and gap

11 DTW was developed to align sequences that express similar patterns at different local rates (Sakoe and Chiba 1978).
12 Standard DTW defines a local cost between elements and uses dynamic programming to find a minimum-cost
13 monotone path through the pairwise cost matrix. A horizontal or vertical step can absorb an insertion, deletion, or local
14 shift, while the path still preserves event order. This makes DTW useful for controller traces: network and timestamp
15 jitter should be alignable, whereas missing, additional, or different event groups should retain a positive cost.

16 The review found studies of scripted controller testing, NTCIP controller interfaces, controller-in-the-loop experi-
17 ments, and high-resolution logger validation. It did not identify a prior evaluation combining (1) long-duration
18 input sequences derived from live field logs, (2) repeated replay across numerous production configurations, and (3)
19 automated alignment of complete output-event traces to detect software- and parameter-induced changes. This is a
20 cautious combination claim, not a claim that any individual technology is new.

Study	Input source	Environment	Output evaluation	Difference from this study
Li et al.; Ahmed et al.	Authored XML/CID-Script tests	Controller and interface device	Predefined expected re-sponses	Field-derived 23-hour tests and whole output traces.
Tung	Authored NTCIP tests	Multiple controller devices	Functional and conformance results	Behavioral regression under recorded operation.
Stevanovic et al.	Microsimulation activity	Six-vendor hardware-in-the-loop	Logger and measure consistency	Field replay and release/configuration comparison.
Wang et al.	Microsimulation activity	Hardware/software/emulator-in-loop	Queue, delay, trajectories	Direct controller-event sequence comparison.

21 Table 1. Relationship to the closest prior controller-testing and controller-in-the-loop work.

22 FIELD-DERIVED REPLAY METHOD

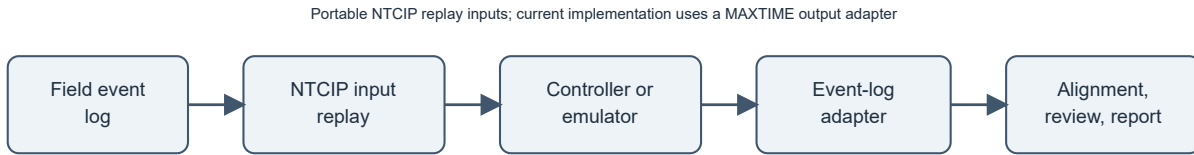


Figure 1: End-to-end workflow. A field log is converted to standard NTCIP input states and replayed to a test controller. The current implementation collects manufacturer-specific output logs, normalizes events, aligns them to a stored baseline, and directs a reviewer to divergences.

23 Field event capture and input reconstruction

24 For each signal configuration, the source log spans approximately 9:00 a.m. to 8:00 a.m. the following day. Across 25
25 Parquet files, the stored source logs contain 7,296,218 high-resolution records and 574.998 configuration-hours. The
26 replay uses the subset representing vehicle detector, pedestrian detector, and preemption state changes. In the evaluated
27 event vocabulary, vehicle detector codes 81 and 82 represent off and on, pedestrian codes 89 and 90 represent off
28 and on, and preemption codes 104 and 102 represent off and on. Parameters 65 and above are excluded as dummy
29 detectors.

30 The software first corrects invalid repeated on/off records by imputing the missing opposite transition. For vehicle and
31 pedestrian channels, detector numbers are packed in groups of eight. Each event changes one bit in an eight-bit state

1 value, and the cumulative state is written to the corresponding controller object. Preemption channels are written as 2 individual Boolean states. Table 2 connects the field record to the replay action.

Input class	Event codes	Field parameter	Replay conversion
Vehicle detector	81 off; 82 on	Detector 1–64	Update its bit in an eight-channel vehicle-detector group and send the cumulative integer state.
Pedestrian detector	89 off; 90 on	Detector 1–64	Update its bit in an eight-channel pedestrian-detector group and send the cumulative integer state.
Preemption	104 off; 102 on	Preempt number	Send 0 or 1 to the corresponding preempt input.

3 Table 2. Conversion from high-resolution field events to controller input states.

4 NTCIP 1202 defines data objects and SNMP-based access for actuated signal controllers (American Association of
5 State Highway and Transportation Officials and Institute of Transportation Engineers and National Electrical Manu-
6 facturers Association 2023). The implementation sends SNMP SET operations to the vehicle-detector-call, pedestrian-
7 detector-call, and preempt-call object groups. It resets all input groups before a replay, then schedules every cumulative
8 state transition at the same time of day as the source record. Thus the input boundary is based on standardized controller
9 objects rather than a proprietary hardware connector.

10 The method is manufacturer-neutral in concept but not plug-and-play across all products. The evaluated implementa-
11 tion used MAXTIME controller emulators, and output was collected through a MAXTIME HTTP event-log interface.
12 Adapting the workflow to another NTCIP controller requires verifying its object implementation, writing an output-
13 collection adapter, and mapping its output codes into the comparison vocabulary. The experiment establishes feasibility
14 on one platform; it does not establish conformance or equivalent behavior for every NTCIP controller.

15 **Test-controller preparation and replay timing**

16 Each emulator was loaded with the production database for the associated signal. Accommodations were needed
17 because an emulator is not a complete field cabinet: detector delay and extension were removed, input/output modules
18 above the first were disabled, inverted preempt logic was removed, peer addresses were set to localhost, and overlap
19 pedestrian inputs were driven by dummy detector mappings. These changes let the software controller receive equiv-
20 alent logical calls without field input hardware. They also limit generalization and must be documented whenever the
21 method is repeated.

22 The replay preserves the original time of day so that time-of-day plans and coordination schedules are exercised.
23 Network transmission and controller logging introduce latency, so raw timestamps are not assumed identical. A
24 configurable offset advances input transmission relative to the desired logged time. In the 2.18.1 campaigns, the runner
25 also estimated latency from matched, isolated detector-on events during replay and updated the offset after a minimum
26 sample count. The stored 2.18.1 release database contains 707,458 latency samples and 6,577 applied updates. This
27 correction addresses transport delay; it does not change the order or intended time spacing of source calls.

28 **Baseline and candidate traces**

29 The initial 2.15.1 replay generated the baseline output trace for every production configuration. When version 2.18.1
30 became available, the same field input sequence was replayed to the same logical configurations and a new output trace
31 was collected. A third campaign again used version 2.18.1, but introduced trailing-yellow, trailing-red, and associated
32 red-revert timing changes where applicable. The three stored databases contain 627,240 replay commands each and
33 approximately 7.6 to 8.0 million controller-output events.

34 Output comparison uses event identifiers representing phase and overlap colors, phase calls, pedestrian service,
35 preemption, coordination transitions, and related state changes. Events occurring at the same timestamp—or within a
36 0.25-second grouping tolerance—form a set of (event identifier, parameter) pairs. Treating simultaneous records
37 as a set avoids interpreting database row order as controller behavior.

38 **DYNAMIC TIME WARPING COMPARISON**

39 **Local cost and warping path**

40 Let $A = (A_1, \dots, A_n)$ and $B = (B_1, \dots, B_m)$ be the ordered reference and candidate event groups. Each A_i or B_j is a
41 set of event-parameter pairs. The implementation uses Jaccard distance as the local mismatch cost:

$$d(A_i, B_j) = 1 - \frac{|A_i \cap B_j|}{|A_i \cup B_j|}.$$

1 Identical groups have cost zero; partially overlapping groups have a cost between zero and one; groups with no common
2 events have cost one. This definition distinguishes a complete group match from a group with missing or additional
3 events while ignoring within-timestamp row order.

4 The cumulative cost matrix is calculated using the standard three-step DTW recurrence:

$$D(i, j) = d(A_i, B_j) + \min(D(i-1, j), D(i, j-1), D(i-1, j-1)),$$

5 with $D(0, 0) = 0$ and inaccessible borders initialized to infinity. Backtracking from $D(n, m)$ gives a monotone
6 warping path. Diagonal steps pair successive groups. Horizontal or vertical steps align one group against more than
7 one group and therefore expose local additions, omissions, or shifts. DTW does not reorder events arbitrarily because
8 both path indices are monotone.

Illustrative event grouping before sequence and timing comparison

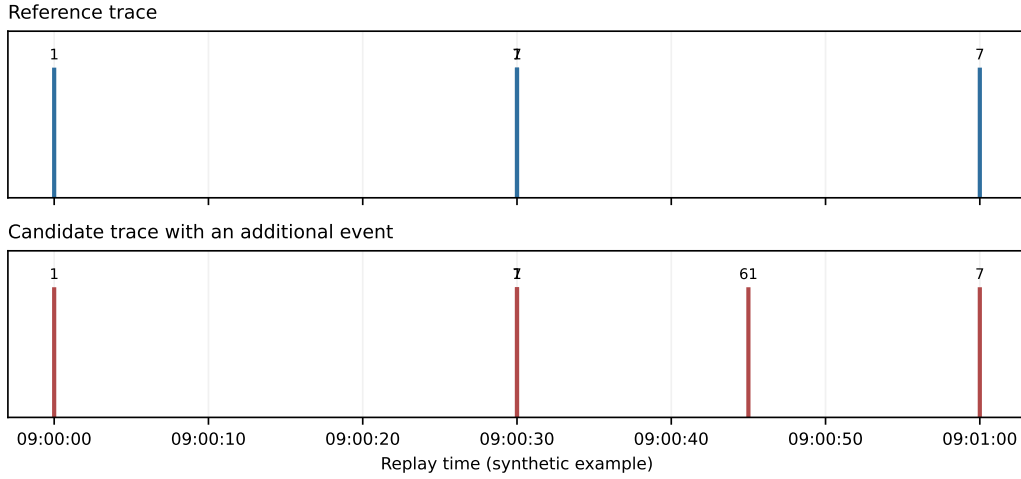


Figure 2: Simplified event-group alignment. DTW can absorb a small timestamp shift while a missing or additional group produces a nonzero Jaccard cost and a horizontal or vertical path segment. The figure is explanatory; reported field results use the full stored traces.

9 Windowed score, timing score, and reliability screening

10 A full-day pair can contain too many groups for an unconstrained quadratic cost matrix. The official sequence score
11 is therefore calculated over 45-minute rolling windows advanced every 40 minutes. DTW sees the full window, but
12 the first and last 60 seconds are excluded from scoring so a window boundary that splits a signal cycle does not create
13 an artificial mismatch. For each scored path, the sequence match is the percentage of aligned group pairs with zero
14 Jaccard distance. The configuration score is the mean of included window scores.

15 The replayed detector calls should also agree between campaigns. Vehicle phase-call similarity is calculated indepen-
16 dently for each window as a simulation-reliability check. A window below 85 percent phase-call similarity remains
17 visible in diagnostics but is excluded from the configuration average. This prevents unreliable controller input response
18 from being reported as a release difference.

19 After sequence alignment, timing is assessed only for exactly matched event groups. For a matched path pair (i, j) , the
20 absolute timing error is $|t(A_i) - t(B_j)|$ after global temporal alignment. Timing match is the percentage of such pairs
21 within 0.50 seconds. The screening thresholds used for the archived results are at least 95 percent sequence match and
22 at least 90 percent timing match. A failure means that a configuration requires review; it is not automatically a software
23 defect.

Parameter	Value	Role
Event grouping tolerance	0.25 s	Combines records representing the same controller instant.
DTW window / advance	45 / 40 min	Bounds computation while retaining five-minute overlap.
Window edge clip	60 s	Avoids scoring a cycle split at a window boundary.
Sequence threshold	95%	Minimum exactly matched aligned groups.

Timing tolerance / threshold	0.50 s / 90%	Minimum share of exact matches within the allowed time error.
Phase-call reliability threshold	85%	Excludes windows in which replay response is not sufficiently comparable.

1 Table 3. Comparison settings used for the reported campaigns.

2 A useful property of this separation is interpretability. Sequence mismatch identifies different controller events; timing
3 mismatch identifies corresponding events occurring at different times. Raw event-count differences alone cannot
4 make that distinction. Diagnostic plots then show the aligned state intervals surrounding each divergence for manual
5 operational classification.

6 EXPERIMENTAL EVALUATION

7 Experiment 1: software release

8 The first evaluation compared baseline software 2.15.1 with candidate software 2.18.1 across the same 25 production
9 configurations and field-derived input sequences. This is the intended operational use: a new release is screened against
10 a prior approved baseline before field deployment. Because software behavior may legitimately change, the outcome
11 is not expected to be universal equality. Feasibility is supported if most long traces align and the exceptions direct the
12 analyst to coherent operational differences.

13 Experiment 2: same-software parameter intervention

14 The second evaluation held software at 2.18.1. Transaction histories identify trailing-overlap, trailing-red, or associated
15 red-revert edits in 14 configurations and no relevant edits in 11. The identical input histories were replayed again. The
16 unchanged group measures repeatability across separate replay campaigns. The changed group is a positive-control
17 intervention: the method should flag configurations only when the recorded traffic sequence activates behavior affected
18 by the edit.

Campaign	Reference	Candidate	Configurations	Purpose
Software release	2.15.1 baseline	2.18.1	25	Release-to-release behavioral screen
Parameter intervention	2.18.1	2.18.1 with trailing-overlap changes	14 changed; 11 unchanged	Repeatability and known-change detection

19 Table 4. Experimental comparisons.

20 RESULTS

21 Repeatability establishes feasibility

22 The strongest evidence that field-event replay is deterministic enough for comparison comes from the 11 configurations
23 that had no relevant parameter change between the two 2.18.1 campaigns. All 11 passed. Their sequence scores ranged
24 from 99.1 to 100.0 percent, with a mean of 99.6 percent. Their timing scores ranged from 90.6 to 99.7 percent, with a
25 mean of 97.7 percent. These are independent full-day replay campaigns, not a comparison of a file with itself.

26 This result answers the first part of the research question: after latency compensation and event grouping, nearly all
27 event groups produced by unchanged software and configuration can be aligned exactly. The residual mismatch is
28 small enough that operational divergences are not hidden by ordinary replay and logging variation.

29 Detection of an operational timing intervention

30 Of the 14 configurations with applicable trailing-overlap or related changes, 11 fell below the comparison thresholds
31 and were flagged. The sequence match among changed configurations averaged 71.9 percent and ranged from 37.8 to
32 100.0 percent. None of the 11 unchanged configurations was flagged. Table 5 presents the resulting detection matrix.

Configuration group	Flagged	Passed	Total
Applicable trailing-overlap change	11	3	14
No relevant change	0	11	11

33 Table 5. Automated outcomes for the same-software parameter-intervention campaign.

34 The three changed configurations that passed are not necessarily algorithmic false negatives. Their sequence matches
35 were 100.0, 99.9, and 99.3 percent. A changed parameter can affect output only when the 23-hour input trace exercises

1 the relevant overlap state and termination condition. The result therefore measures detection under the recorded field
 2 exposure, not sensitivity to every possible timing edit.

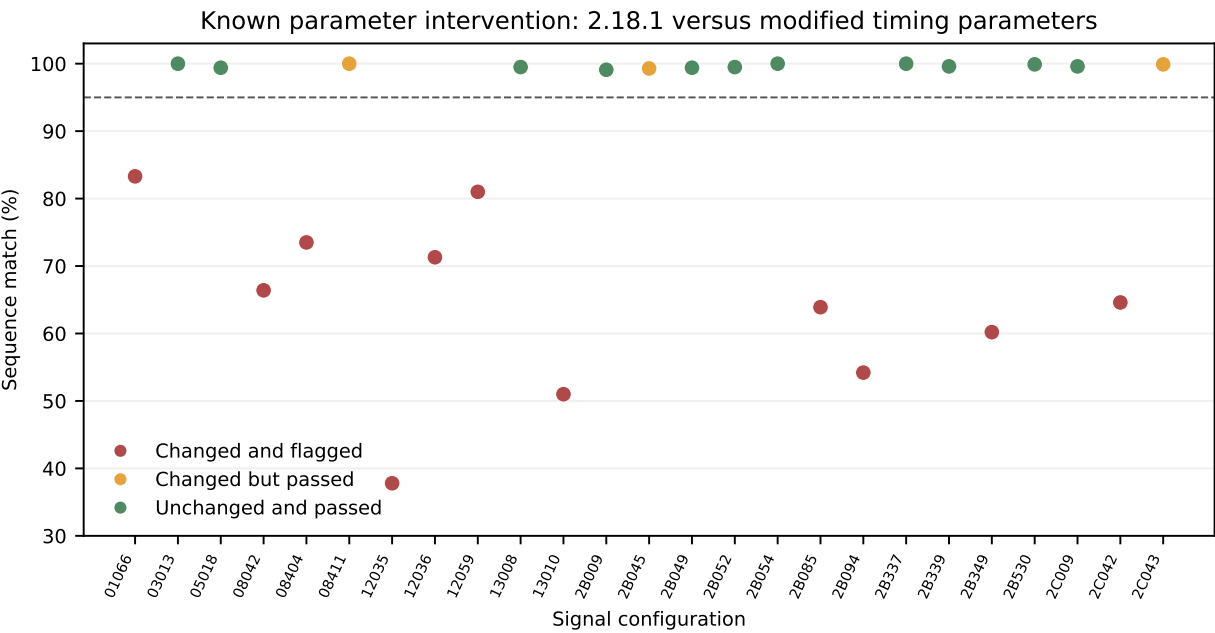


Figure 3: Parameter-intervention results by configuration. Eleven changed configurations were automatically flagged; three changed configurations passed because little or no affected behavior appeared in the replay; all 11 configurations without relevant edits passed.

3 The diagnostic plot in Figure 4 shows the operational effect that motivated the intervention. Under the earlier settings,
 4 short and variable overlap-yellow intervals appeared in the output. Adding trailing-yellow and trailing-red behavior
 5 removed the short intervals while leaving other operation substantially similar. This is precisely the intended use of
 6 regression replay: confirm that a targeted timing change affects the problematic behavior without introducing broad,
 7 unrelated changes.

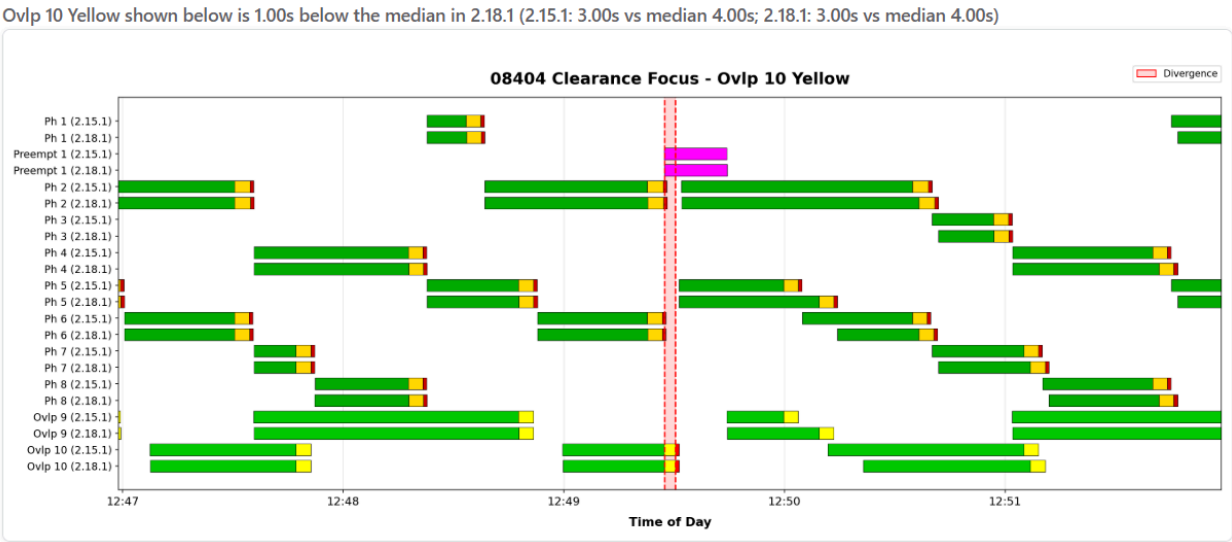


Figure 4: Representative overlap-clearance diagnostic from the parameter study. The aligned event traces make short overlap-yellow intervals visible and show their removal after trailing-yellow and trailing-red settings were applied.
 Source: project diagnostic generated from the stored controller-output events.

1 Software-release screening

2 The 2.15.1-to-2.18.1 comparison passed 19 of 25 configurations. Mean sequence match across the configuration
3 summaries was 96.2 percent, and mean reported timing match was 93.8 percent. Six configurations required review:
4 12036, 2B045, 2B054, 2B085, 2B094, and 2B339. Four failed the 95-percent sequence criterion only, and two exceeded
5 the sequence criterion but fell below the 90-percent timing criterion.

ID	Seq.	Time	Result	ID	Seq.	Time	Result
01066	96.1	91.9	Pass	03013	99.0	90.8	Pass
05018	97.9	98.2	Pass	08042	98.6	96.5	Pass
08404	99.4	96.5	Pass	08411	98.6	95.7	Pass
12035	95.3	90.7	Pass	12036	75.4	—	Review
12059	99.1	98.4	Pass	13008	98.6	96.6	Pass
13010	98.9	90.1	Pass	2B009	97.9	92.4	Pass
2B045	87.7	—	Review	2B049	95.4	92.6	Pass
2B052	98.6	92.0	Pass	2B054	98.7	89.0	Review
2B085	95.9	87.6	Review	2B094	94.8	—	Review
2B337	99.9	95.0	Pass	2B339	84.5	—	Review
2B349	97.9	93.7	Pass	2B530	99.4	95.6	Pass
2C009	97.6	92.1	Pass	2C042	99.7	96.1	Pass
2C043	99.7	98.6	Pass				

6 Table 6. Version 2.15.1 versus 2.18.1 sequence and timing match percentages. Timing is not reported when sequence
7 match is below 95 percent.

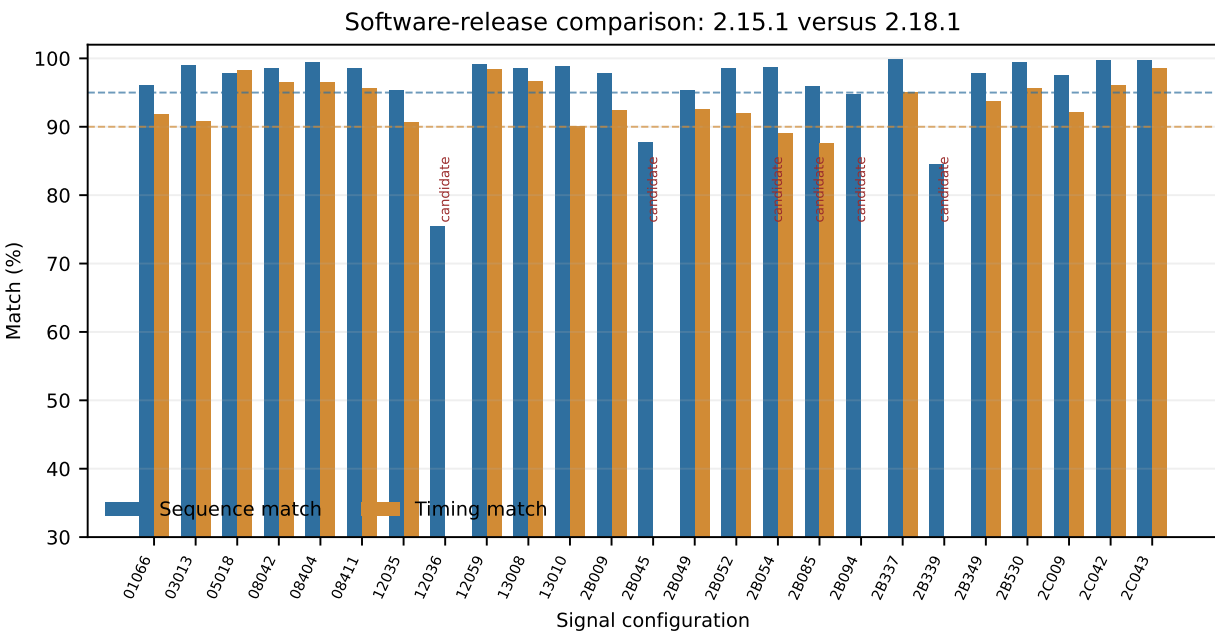


Figure 5: Software-release comparison across 25 production configurations. Most output traces exceeded both thresh-
olds, demonstrating broad alignment; six configurations were directed to manual review.

8 A review candidate can be a defect, an intentional change, a bug fix, or an unexplained difference. Manual review
9 of the release diagnostics found examples of each useful classification except a confirmed new operation-impacting
10 defect. Most notably, version 2.15.1 exhibited a rail-preemption exit behavior that had previously been identified as
11 a bug; version 2.18.1 no longer produced that sequence. The automated comparator correctly reported the traces as
12 divergent. In this case, a failed regression threshold is positive evidence that the expected bug fix changed operation.

Preempt 6 Active: largest local mismatch window; mismatch 790.40s; 2.15.1 events 1, active 790.40s; 2.18.1 events 0, active 0.00s

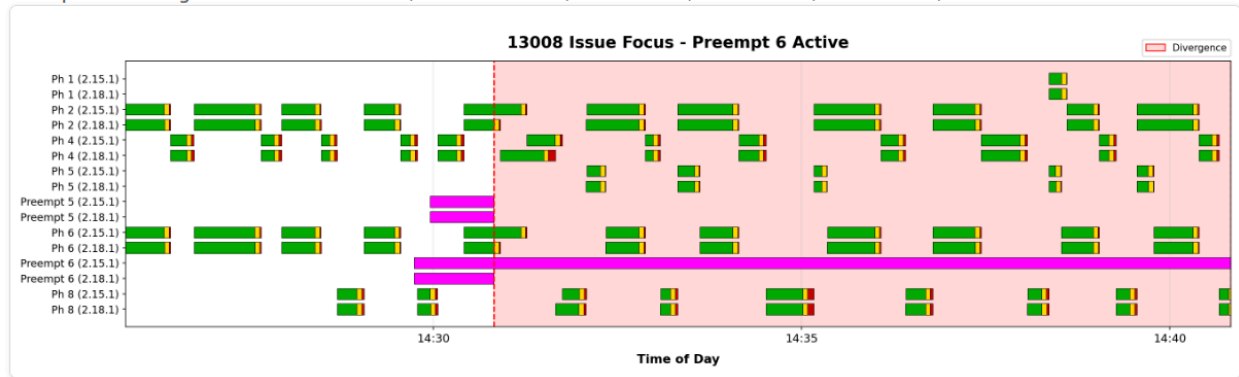


Figure 6: Representative aligned preemption-exit trace. The divergence corresponds to a previously identified version 2.15.1 rail-preemption exit defect that was corrected in version 2.18.1. The figure illustrates why automated failures require operational classification rather than being labeled defects automatically.

- 1 The release also changed recorded transition-state behavior. In the representative example in Figure 7, the controller's
- 2 internal transition state differed while the observed signal operation did not show a material effect. This is still valuable
- 3 information: the method detected a software-state change that a reviewer could classify as non-operational rather than
- 4 silently assuming release equivalence.

Transition Longway: largest local mismatch window; mismatch 917.50s; 2.15.1 events 1, active 686.40s; 2.18.1 events 1, active 231.10s; window avg 2.15.1 686.40s vs 2.18.1 231.10s

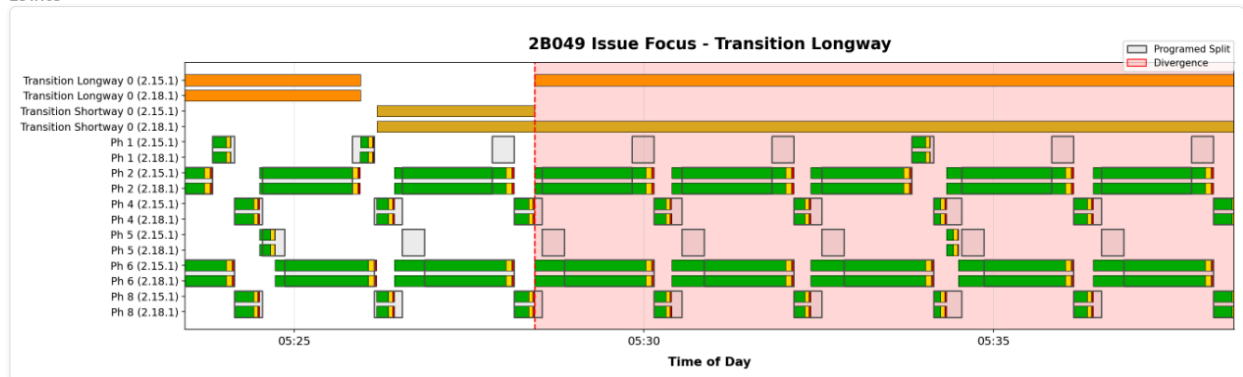


Figure 7: Representative coordination-transition difference. The high-resolution states differ between releases, but project review found no noticeable operational consequence in the displayed interval.

- 5 Taken together, the software-release campaign supports the second part of the feasibility question. DTW aligned most
- 6 of each 23-hour output trace while preserving localized changes associated with a bug fix and transition logic. The
- 7 method reduced millions of event records to a short list of configurations and time intervals requiring engineering
- 8 review.

9 DISCUSSION

10 Why field-derived replay is useful

11 A field trace is not a synthetic demand profile. It contains the actual ordering and subsecond timing of calls experienced
 12 by a production controller, including simultaneous events and rare combinations that may be difficult to anticipate in
 13 a test plan. Replaying 23 hours across 25 configurations therefore exercises substantially more controller logic than a
 14 person can reproduce manually. It also preserves realistic correlation among inputs: pedestrian calls occur alongside
 15 vehicle calls, coordination transitions occur at scheduled times, and preemption interrupts the prevailing state.

16 The method complements rather than replaces authored tests. A scripted functional test is preferable when a require-
 17 ment has a precise expected response or when safety logic must be exhaustively challenged. Field-derived replay is
 18 preferable as a broad regression screen and as a way to reproduce an observed incident. If a field controller flashes
 19 or behaves unexpectedly, the events preceding the occurrence can be extracted, replayed repeatedly, and shortened

1 to isolate a triggering sequence. The same test can then verify a vendor software correction or an agency timing
2 workaround.

3 A baseline trace also changes the role of expected output. An engineer need not specify millions of individual responses.
4 The accepted controller run supplies an executable behavioral oracle, while DTW accommodates benign subsecond
5 variability. This makes detailed regression practical, but it also means that baseline defects can be preserved. A baseline
6 should therefore be an approved version, and detected improvements—as in the preemption fix—must be deliberately
7 accepted into a new baseline.

8 **Interpretation of automated flags**

9 The comparison is a triage system, not an automatic safety certification. A sequence failure indicates different event
10 content; a timing failure indicates that matched states occurred at different times. Neither alone explains the operational
11 cause. Review should consider the aligned interval, relevant timing parameters, controller release notes, and whether
12 the field input exercised the intended condition.

13 The same-software experiment illustrates both sensitivity and exposure. It produced no flags among unchanged
14 configurations and large divergences in 11 changed configurations, but three applicable changes produced virtually
15 identical output. A regression suite built from field operation gains realism at the cost of controlled coverage. Agencies
16 can address that limitation by accumulating traces from multiple days, retaining targeted incident traces, and supple-
17 menting the field corpus with authored calls for requirements not observed in the logs.

18 **Portability**

19 NTCIP 1202 is an important portability boundary because the input states do not depend on a physical suitcase
20 connector or a proprietary simulation API. Nevertheless, NTCIP conformance is not sufficient by itself. Products may
21 implement optional objects differently, timestamp output differently, or expose high-resolution logs through different
22 mechanisms. Before cross-vendor use, the replay adapter, output adapter, event mapping, and latency model must be
23 validated against known calls.

24 The current output collector is manufacturer-specific, but the comparison is not. Once two controller traces are
25 converted to the common (event identifier, parameter, timestamp) representation, the same grouping, DTW
26 alignment, timing analysis, and reporting code can run offline. This separation makes a multi-vendor extension
27 technically plausible and identifies exactly what integration work remains.

28 **LIMITATIONS**

29 The evaluation used software emulators from one controller product family. Emulator behavior may differ from
30 physical controller hardware, especially where cabinet inputs, hardware monitoring, or operating-system timing are
31 involved. Configuration accommodations made for the emulator could also influence behavior. Replication on physical
32 controllers and a second controller family is needed before claiming general cross-platform validity.

33 The field records represent one agency, 25 configurations, and one approximately 23-hour period per configuration.
34 The traces are broad but not exhaustive. A behavior absent from the recorded day cannot be evaluated, which is the
35 likely explanation for at least some changed configurations that passed the parameter intervention. Future work should
36 quantify incremental coverage from additional days and targeted incident traces.

37 The reported thresholds were engineering screening settings, not statistically optimized decision boundaries. The
38 present archive does not contain completed one-factor sensitivity reruns for grouping tolerance, phase-call reliability
39 threshold, and timing tolerance. Those analyses remain a high-value pre-submission improvement. The six release
40 review candidates also require a configuration-by-configuration final classification by the author; the manuscript
41 currently reports only the manually reviewed bug-fix and transition examples supported by the project presentation
42 and diagnostic figures.

43 The original controller replay requires emulator software, production configurations, and agency infrastructure. The
44 stored databases allow the comparison and reporting stage to be reproduced without those systems, but the replay
45 campaign itself is only procedurally reproducible. Finally, a prior baseline can contain unwanted behavior. Automated
46 similarity cannot decide whether the reference behavior is correct.

47 **CONCLUSIONS**

48 This study provides a positive feasibility result. Approximately 23 hours of field-derived vehicle, pedestrian, and
49 preemption events for each of 25 production signal configurations were converted to NTCIP calls and replayed to test

1 controllers. A Jaccard-cost DTW comparison aligned the resulting discrete event groups despite small timing variations
2 and automatically retained missing, additional, and changed events as divergences.

3 The same-software comparison supplies the clearest evidence of determinism: all 11 unchanged configurations passed,
4 with 99.6 percent mean sequence match and 97.7 percent mean timing match. The known timing intervention was
5 flagged in 11 of 14 affected configurations and in none of the unchanged configurations. The version 2.15.1-to-2.18.1
6 campaign passed 19 configurations and directed six to review, including an expected preemption bug fix and a
7 transition-state change.

8 The practical finding is not that replay makes controller testing automatic from end to end. It makes an otherwise
9 unmanageable quantity of real operational testing repeatable and reviewable. Field histories can become a regression
10 corpus for release screening, timing-change evaluation, and bug replication. Engineering judgment remains necessary
11 to approve the baseline, interpret divergences, and determine whether a detected difference is beneficial, harmful, or
12 immaterial.

13 **ACKNOWLEDGMENTS**

14 The author developed and evaluated this work as part of traffic-signal operations at the Oregon Department of
15 Transportation. [AUTHOR REVIEW: add any colleagues, vendor personnel, funding, or required agency disclaimer
16 before submission.]

17 **CODE AND DATA AVAILABILITY**

18 Signal-Replay is an open-source Python package available at [https://github.com/ShawnStrasser/ATC-Signal-](https://github.com/ShawnStrasser/ATC-Signal-Replay)
19 [Replay](https://github.com/ShawnStrasser/ATC-Signal-Replay). The repository includes the replay and comparison implementation, tests, an offline synthetic example,
20 analysis scripts, and instructions distinguishing controller-dependent replay from offline comparison. The exact public
21 release or commit cited by the submitted paper must be added after the author pushes the current work. A candidate
22 export script can package replay inputs, collected outputs, parameters, and expected summaries without controller
23 firmware or proprietary configuration databases. No dataset has been published automatically.

24 **GENERATIVE-AI DISCLOSURE**

25 OpenAI Codex was used to inspect repository materials, assist with literature discovery, revise analysis and documen-
26 tation code, generate data visualizations from stored results, and draft and format the manuscript. The author reviewed
27 the source materials and is responsible for the methods, factual accuracy, interpretation, citations, and submitted text.

28 **REFERENCES**

29 Ahmed, Sk. Monsur, Cody Browne, Ahmed Abdel-Rahim, and Richard Wall. 2010. *An Automated Testing Tool for*
30 *Traffic Signal Controller Functionalities*. Technical Report FHWA-ID-10-180. [https://rosap.ntl.bts.gov/view/dot/](https://rosap.ntl.bts.gov/view/dot/23916)
31 [23916](https://rosap.ntl.bts.gov/view/dot/23916).

32 American Association of State Highway and Transportation Officials and Institute of Transportation Engineers and
33 National Electrical Manufacturers Association. 2023. “NTCIP 1202 V03b: Object Definitions for Actuated Traffic
34 Signal Controllers Interface.”. <https://www.ntcip.org/document-numbers-and-status/>.

35 Li, Zhen, Ahmed Ahmed-Abdel-Rahim, Brian Johnson, and Michael Kyte. 2008. “Design of Traffic Controller
36 Automated Testing Tool.” *Transportation Research Part C: Emerging Technologies* 16 (3): 277–93. [https://doi.](https://doi.org/10.1016/j.trc.2007.10.001)
37 [org/10.1016/j.trc.2007.10.001](https://doi.org/10.1016/j.trc.2007.10.001).

38 Sakoe, Hiroaki, and Seibi Chiba. 1978. “Dynamic Programming Algorithm Optimization for Spoken Word Recog-
39 nition.” *IEEE Transactions on Acoustics, Speech, And Signal Processing* 26 (1): 43–49. [https://doi.org/10.1109/](https://doi.org/10.1109/TASSP.1978.1163055)
40 [TASSP.1978.1163055](https://doi.org/10.1109/TASSP.1978.1163055).

41 Stevanovic, Aleksandar, Ivica Klanac, and Danilo Radivojevic. 2017. *Development of Minimum Standards for Event-*
42 *Based Data Collection Loggers and Performance Measure Definitions for Signalized Intersections*. Technical
43 Report Nos. BDV27–977–6. <https://rosap.ntl.bts.gov/view/dot/31807>.

44 Sturdevant, James R., Tim Overman, Eric Raamot, et al. 2012. *Indiana Traffic Signal Hi Resolution Data Logger*
45 *Enumerations*. Technical report. <https://doi.org/10.4231/K4RN35SH>.

46 Tung, Leonard. 2012. *Development of Automated Testing Tools for Traffic Control Signals and Devices (NTCIP and*
47 *Security)*. Technical report. <https://rosap.ntl.bts.gov/view/dot/25053>.

- 1 Tung, Leonard. 2015. *Development of Automated Testing Tools for Traffic Control Signals and Devices (NTCIP and*
2 *Security), Phase 2*. Technical Report Nos. BDV30–977–5. <https://rosap.ntl.bts.gov/view/dot/28634>.
- 3 Wang, Daobin, Zongbei Tian, and Guangchuan Yang. 2021. “Virtual Controller Interface Device for Hardware-in-the-
4 Loop Simulation of Traffic Signals.” *IEEE Intelligent Transportation Systems Magazine* 13 (2): 201–16. [https://](https://doi.org/10.1109/MITS.2019.2898968)
5 doi.org/10.1109/MITS.2019.2898968.